# Statement Portal — Frontend Integration Guide

Every endpoint this API exposes, with headers, request/response bodies, and error shapes, for wiring up the frontend without needing to read the backend source.

---

This API has five endpoints in total, listed below. All of them require a bearer token already issued by LotusBank SSO — this service validates that token itself and does not provide its own login. How your app obtains that token from SSO in the first place is outside the scope of this document; ask the SSO team for their frontend integration contract. Everything from "I have a token" onward is covered here.

| Method | Path | Purpose |
|---|---|---|
| GET | `/api/me` | Current user's identity + permissions (drives what UI to show) |
| POST | `/api/statements/single` | Generate one customer statement |
| POST | `/api/statements/bulk` | Submit a batch of statement requests |
| GET | `/api/audit` | Search/filter audit records (paginated) |
| GET | `/api/audit/{id}` | Full detail for one audit record |

# 1. Authentication — required on every request

There are two ways to authenticate a request, and both work with the same token:

| Option | How |
|---|---|
| Authorization header (recommended) | Send `Authorization: Bearer <token>` on every request. Works regardless of domain/cookie setup. |
| Cookie | If your app runs on a domain that shares SSO's "access-token" cookie, the browser sends it automatically — but only if your fetch/XHR call is made with `credentials: 'include'` (fetch) or `withCredentials: true` (XHR/axios), and your app's origin is on the API's CORS allow-list (ask the backend team to confirm it's listed). |

> **Watch out:** If both are absent or the token is invalid/expired, every endpoint returns `401 Unauthorized` with an empty body (see Section 3.3) — the browser console will show a 401, not a redirect to a login page. Your app is responsible for detecting 401s and sending the user back through SSO's login flow.

### What's in the token that matters to the frontend

The token carries the staff member's permissions for this application. You don't need to decode the token yourself to know what the user can do — call `GET /api/me` (Section 4) once after login and use its `permissions` array to decide what to render.

> **Note:** Showing or hiding a button based on `/api/me`'s permissions is a display convenience only. The backend independently re-checks the required permission on every protected endpoint — a hidden button is not a security control, so don't rely on the UI hiding something as the reason it's "safe" to call that endpoint from elsewhere in the app.

---

# 2. Conventions used across every endpoint

## 2.1 Base URL

All paths below are relative to the API's base URL for the environment you're pointed at (e.g. `https://statements.lotusbank.internal` or your local `https://localhost:xxxx`). Confirm the exact value with the backend team per environment.

## 2.2 Headers on every request

| Header | Value | Required |
|---|---|---|
| Authorization | Bearer <token> | Yes, unless using the cookie (Section 1) |
| Content-Type | application/json | Yes, on POST requests (single/bulk statements) |

| Header | Value | Required |
|---|---|---|
| Accept | application/json | Recommended |

### 2.3 JSON casing and data types

- All JSON field names are **camelCase** (e.g. `accountNumber`, not `AccountNumber`).
- Dates (`startDate`, `endDate`, `date`) are date-only strings in `yyyy-MM-dd` format — no time component, no timezone. Send them the same way in request bodies and query strings.
- `time` fields are `HH:mm:ss`.
- GUIDs (`requestId`) are standard hyphenated strings, e.g. "3fa85f64-5717-4562-b3fc-2c963f66afa6".
- Enums (`status`) are sent and returned as their string name (e.g. "Success", not a number).

### 2.4 Pagination

`GET /api/audit` is the only paginated endpoint. Request `page` (1-based) and `pageSize` as query parameters; the response tells you the total count so you can compute total pages (`Math.ceil(totalCount / pageSize)`). `pageSize` is capped server-side at 100 regardless of what you request.

### 2.5 Rate limiting

`/api/statements/*` is limited to **30 requests per minute per user**. Exceeding it returns `429 Too Many Requests` with an empty body — surface a generic "please slow down / try again shortly" message rather than treating it as a hard failure.

### 2.6 CORS gotcha

> **Watch out:** Your app's exact origin (scheme + host + port) must be on the backend's allow-list. A mismatch (e.g. testing on a different port than what's registered) fails silently from your code's perspective — the browser blocks the response before your JS ever sees a status code, and it shows only in the browser console as a CORS error, not as a catchable HTTP error in most fetch wrappers. If requests seem to vanish with no response at all, check the console for a CORS message before assuming the backend is down.

## 3. Error response reference

Response shapes are **not uniform** across error types — this is the single most important thing to get right in your error-handling code, so read this before wiring up each endpoint.

| Status | When | Body shape |
|---|---|---|
| 400 Bad Request | Validation failure on a statement request (Section 5/6) | `{ "errors": ["message", ...] }` |
| 401 Unauthorized | Missing/invalid/expired token | Empty body |
| 403 Forbidden | Valid token, but missing the required permission | Empty body |
| 404 Not Found | GET /api/audit/{id} — record doesn't exist | RFC 7807 ProblemDetails, e.g. `{ "title": "Not Found", "status": 404, "traceId": "..." }` |
| 429 Too Many Requests | Rate limit exceeded on /api/statements/* | Empty body |
| 500 Internal Server Error | Unhandled server-side error | `{ "message": "An unexpected error occurred. Please try again or contact support." }` |

> **Note:** For 400 responses specifically, `errors` is a flat array of human-readable strings (not field-keyed) — good enough to show directly to the user in most cases, but you cannot map a given message back to a specific form field programmatically.

> **Watch out:** For 401/403/429, write your HTTP client wrapper to branch on **status code alone** — don't assume there's a JSON body to parse, or your error handler itself will throw trying to parse an empty response.

## 4. Endpoint reference

**GET** `/api/me`

Required policy: **any authenticated user** • Call once after login to get identity + permissions.

Use this to populate the app's user context and decide which modules/menu items to render.

### Request

No body, no query parameters. Just the Authorization header from Section 1.

```
GET /api/me
Authorization: Bearer eyJhbGciOi...
```

## Response — 200 OK

```
{
  "staffId": "10234",
  "staffName": "Ada Okafor",
  "email": "ada.okafor@lotusbank.com",
  "permissions": ["SingleStatement", "BulkStatement"]
}
```

| Field | Type | Notes |
|---|---|---|
| staffId | string | From the token's subject claim. |
| staffName | string | May be empty string if SSO's token doesn't carry a name claim. |
| email | string | May be empty string similarly. |
| permissions | string[] | One or more of: "SingleStatement", "BulkStatement", "Audit", "AuditAdmin". An empty array means the user has no access to any module in this app — show an appropriate "not provisioned" state rather than a blank screen. |

## Sample fetch

```
const res = await fetch(`${API_BASE}/api/me`, {
  headers: { Authorization: `Bearer ${token}` }
});
if (res.status === 401) { /* send user back through SSO login */ }
const me = await res.json();
// me.permissions.includes("Audit") -> show Audit nav item
```

## POST /api/statements/single

Required policy: **SingleStatement** • Generate one customer's statement for a date range.

### Request headers

| Header | Value |
|---|---|
| Authorization | Bearer <token> |
| Content-Type | application/json |

### Request body

```
{
  "accountNumber": "1234567890",
  "startDate": "2026-01-01",
  "endDate": "2026-06-30",
  "format": "PDF"
}
```

| Field | Type | Required | Rules |
|---|---|---|---|
| accountNumber | string | Yes | Exactly 10 digits. |
| startDate | string (date) | Yes | yyyy-MM-dd; must not be after endDate. |
| endDate | string (date) | Yes | yyyy-MM-dd; cannot be in the future; range from startDate cannot exceed 12 months. |
| format | string | No — defaults to "PDF" | "PDF" or "CSV" only. |

### Response — 200 OK

```
{
  "requestId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "status": "Success",
  "downloadUrl": "https://.../statements/3fa85f64.../download"
}
```

| Field | Type | Notes |
|---|---|---|
| requestId | string (guid) | Generated by the backend — you never send one. Useful for support/troubleshooting references and for cross-checking against the audit trail. |
| status | string | Reflects the downstream Statement Service's result string. |
| downloadUrl | string \| null | May be null depending on how the Statement Service returns the finished document — confirm with the backend team whether your environment returns a URL to fetch separately or the file is delivered another way. |

### Response — 400 Bad Request (validation failure)

```
{ "errors": ["Account number must be a 10-digit account number.", "End date cannot be in the future."] }
```

> **Note:** A validation failure here means the Statement Service was never called at all — nothing to retry, just show the messages and let the user fix the form.

### Sample fetch

```
const res = await fetch(`${API_BASE}/api/statements/single`, {
  method: 'POST',
  headers: {
    Authorization: `Bearer ${token}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    accountNumber: '1234567890',
    startDate: '2026-01-01',
    endDate: '2026-06-30',
    format: 'PDF'
  })
});
if (res.status === 400) { const { errors } = await res.json(); /* show errors */ }
else if (res.status === 403) { /* user lacks SingleStatement permission */ }
else if (res.ok) { const result = await res.json(); }
```

## POST /api/statements/bulk

Required policy: **BulkStatement** • Submit up to 500 statement requests in one call.

### Request headers

| Header | Value |
|---|---|
| Authorization | Bearer <token> |
| Content-Type | application/json |

### Request body

```
{
  "items": [
    { "accountNumber": "1234567890", "startDate": "2026-01-01", "endDate": "2026-06-30" },
    { "accountNumber": "9876543210", "startDate": "2026-01-01", "endDate": "2026-06-30" }
  ],
  "format": "PDF"
}
```

| Field | Type | Required | Rules |
|---|---|---|---|
| items | array | Yes | 1 to 500 items. Each item needs accountNumber (10 digits), startDate, endDate (same rules as the single endpoint, minus the 12-month cap). |
| format | string | No — defaults to "PDF" | "PDF" or "CSV" only, applies to the whole batch. |

### Response — 200 OK

```
{
  "requestId": "6b2f1a90-....",
  "status": "Accepted",
  "acceptedCount": 2,
  "rejectedCount": 0
}
```

This is a submission acknowledgement, not the finished statements — bulk processing happens asynchronously on the Statement Service's side. Ask the backend team how completed bulk results are surfaced in your environment (polling, a callback, a separate status endpoint) if your UI needs to show per-batch progress; that is not one of this API's current endpoints.

### Response — 400 Bad Request

```
{ "errors": ["A bulk request cannot contain more than 500 items.", "Start date must not be after end date."] }
```

### Sample fetch

```
const res = await fetch(`${API_BASE}/api/statements/bulk`, {
  method: 'POST',
  headers: {
    Authorization: `Bearer ${token}`,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    items: [{ accountNumber: '1234567890', startDate: '2026-01-01', endDate: '2026-06-30' }],
    format: 'PDF'
  })
});
```

## GET `/api/audit`

Required policy: **Audit** • Search/filter the audit trail. All filters are optional and ANDed together.

### Query parameters (all optional)

| Param | Type | Example | Notes |
|---|---|---|---|
| startDate | date | 2026-01-01 | Inclusive. |
| endDate | date | 2026-01-31 | Inclusive. |
| staffNameOrId | string | Ada | Partial match against staff name OR staff id. |
| branch | string | Lagos-Main | Exact match. |
| module | string | SingleStatement | "SingleStatement" or "BulkStatement". |
| activity | string | GenerateStatement | e.g. "GenerateStatement", "SubmitBulkStatement". |
| status | string | Success | "Success", "Failed", or "Denied". |
| page | int | 1 | Defaults to 1. |
| pageSize | int | 25 | Defaults to 25; capped at 100 server-side. |

### Sample request

```
GET /api/audit?startDate=2026-01-01&endDate=2026-01-31&module=SingleStatement&status=Failed&page=1&pageSize=25
Authorization: Bearer eyJhbGciOi...
```

### Response — 200 OK

```
{
  "items": [
    {
      "id": 4821,
      "requestId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "staffName": "Ada Okafor",
      "staffId": "10234",
      "module": "SingleStatement",
      "email": "ada.okafor@lotusbank.com",
      "branch": "Lagos-Main",
      "activity": "GenerateStatement",
      "maskedAccountNumber": "******7890",
      "statementDateRange": "2026-01-01 to 2026-06-30",
      "date": "2026-01-15",
      "time": "14:22:05",
      "status": "Success",
      "ipAddress": "10.0.4.12"
    }
  ],
  "page": 1,
  "pageSize": 25,
  "totalCount": 137
}
```


**Note:** Account numbers are always masked server-side (last 4 digits only) — there is no way to request an unmasked value from this API. Do not build a UI feature that assumes one is available.

## GET /api/audit/{id}

Required policy: **Audit** • Full detail for one audit record.

### Request

```
GET /api/audit/4821
Authorization: Bearer eyJhbGciOi...
```

### Response — 200 OK

Same shape as one item from the search endpoint, plus one extra field:


```
{
  "id": 4821,
  "requestId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "staffName": "Ada Okafor",
  "staffId": "10234",
  "module": "SingleStatement",
  "email": "ada.okafor@lotusbank.com",
  "branch": "Lagos-Main",
  "activity": "GenerateStatement",
  "maskedAccountNumber": "******7890",
  "statementDateRange": "2026-01-01 to 2026-06-30",
  "date": "2026-01-15",
  "time": "14:22:05",
  "status": "Failed",
  "ipAddress": "10.0.4.12",
  "failureReason": "Statement Service returned an empty response."
}
```


| Field | Type | Notes |
|---|---|---|
| failureReason | string \| null | Only meaningful when status is "Failed" or "Denied"; null on a successful activity. |

### Response — 404 Not Found


```
{ "type": "https://tools.ietf.org/html/rfc7231#section-6.5.4", "title": "Not Found", "status": 404 }
```

## 9. Quick integration checklist

1. Confirm the API base URL for your target environment with the backend team.
2. Confirm your app's exact origin is on the CORS allow-list before writing a single line of fetch code — a mismatch here is the most common "nothing works and there's no error" issue.
3. Decide header vs. cookie auth (Section 1) and implement a single HTTP client wrapper that attaches it consistently, rather than repeating it per call site.
4. Handle 401 globally: redirect through SSO's login flow, don't just show a generic error.
5. Handle 403 as "you don't have this permission" — pair it with what /api/me already told you, so this should mostly be a defensive case, not the primary way your UI learns about permissions.
6. Handle 400 by rendering the errors array; handle 429 with a "please slow down" message; treat 500 as a hard failure with retry guidance to the user.
7. Send all dates as yyyy-MM-dd strings, not JavaScript Date objects or ISO datetime strings with a time component.
8. For /api/audit, implement pagination using totalCount from the response rather than assuming a fixed page count.

---

*Generated to accompany the StatementPortal solution on branch claude/batch-reversal-auth-optimize-67ewy3. See ARCHITECTURE.md for backend design rationale.*